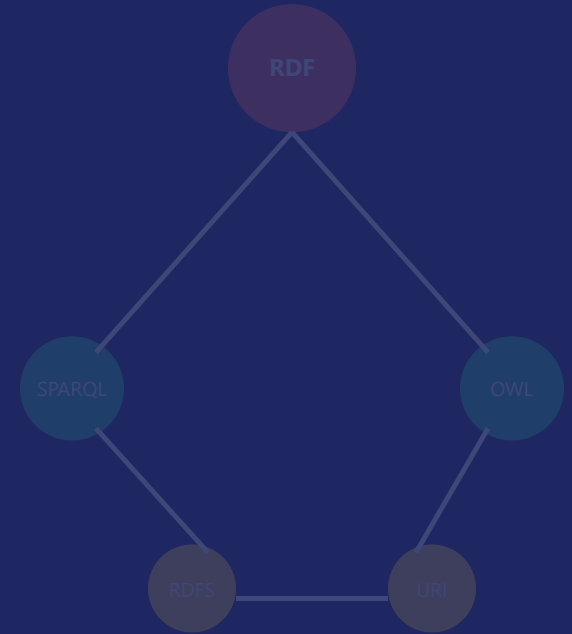


ADVANCED DATA MODELS · SS 2026 · GRUPPE 2

Semantic Triple Stores

RDF · SPARQL · Inference

Roman Lahm & Vinzent Lahm



1

MOTIVATION

Warum Triple Stores? · SQL-
Grenzen · RDF · AI

4

QUERY MODEL

SPARQL · Pattern Matching ·
Inferenz

>_

DEMO

2

CONCEPTUAL MODEL

Triple · URIs · RDFS/OWL ·
Inferenz

5

ARCHITECTURE

Storage · Deployment ·
Skalierung · Tools

**INTERAKTION**

3

DATA STRUCTURE

Triple Table · Dictionary · Indexe

6

COMPARISON

vs. SQL · vs. Property Graph ·
Matrix

**QUIZ**

WIE ALLES BEGANN

**1989 erfindet ein Physiker am
CERN
das _____.**



Sir Tim Berners-Lee

* 1955 · Großbritannien

DER ERFINDER & SEINE IDEE

1989

Proposal „**Information Management**“ — Hyperlinks zwischen Dokumenten

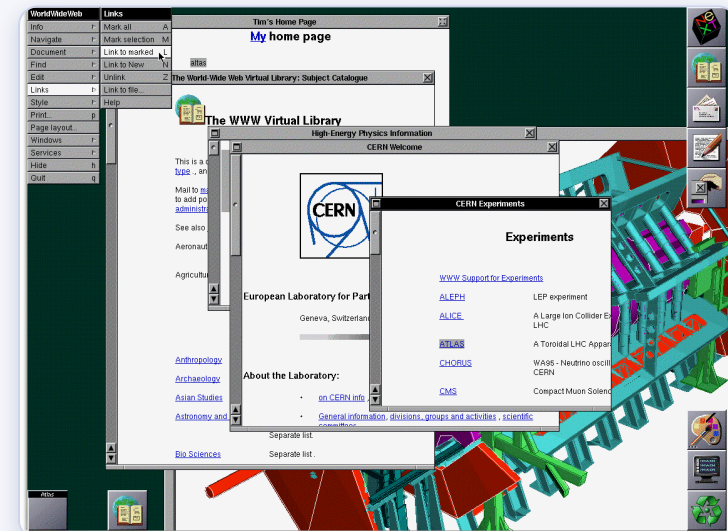
1991

Erstes WWW online — HTML-Seiten, Hyperlinks, Browser

Das Web verbindet **Dokumente** — aber Dokumente haben keine maschinenlesbare Bedeutung.



Sir Tim Berners-Lee · * 1955 · Großbritannien



Erstes WWW — WorldWideWeb-Browser, CERN 1991

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 ARCHITECTURE

6 COMPARISON

WORUM ES IN DIESEM KAPITEL GEHT

Das WWW & sein blinder Fleck – Dokumente verlinkt, ihre Bedeutung nicht



Grenzen von SQL – starre Schemata, isolierte Datensilos



RDF-Grundidee – Wissen als Graph aus einzelnen Fakten



Semantic Web – Maschinen sollen Bedeutung *verstehen*

1998

Geburtsstunde des **Semantic Web** → Grundstein für **RDF** & Triple Stores

DAS PROBLEM: INKOMPATIBLE DATENSILOS



SQL-Tabelle

starres Schema



XML-Datei

eigene Struktur



Excel-Liste

kein Standard

Kein gemeinsames Modell → **Lösung: RDF & Triples** als universelle Sprache.

SQL — BREITE, DÜNN BESETZTE TABELLE

Uni	Stadt	Gegr.	Mensa	Twitter
UniS	Stuttgart	NULL	NULL	@uni_s
KIT	Karlsruhe	1825	Ja	NULL

Jedes neue Attribut = neue Spalte + viele **NULLs**.

RDF — NUR DIE FAKTEN, DIE ES GIBT

```
UniS hasCity "Stuttgart" .
UniS twitter "@uni_s" .
KIT hasCity "Karlsruhe" .
KIT founded 1825 .
KIT hasMensa true .
→ kein NULL, kein ALTER TABLE
```

2001**Scientific American, Mai 2001**

„The Semantic Web“ — Berners-Lee, Hendler, Lassila

WAS BEDEUTET SEMANTIK?

Semantik = die **Bedeutung** hinter Zeichen und Symbolen

✗ SQL: `company = "SAP"` — nur ein String

✓ RDF: `SAP rdf:type Company` — maschinenlesbare Bedeutung

PETE & LUCY — EINE VISION

SZENARIO

Pete & Lucy suchen einen **Physiotherapeuten** für ihre Mutter.

KLASSISCH

Alle Therapeuten der Umgebung manuell durchgehen & mit dem **Terminkalender der Mutter** abgleichen.

MIT SEMANTIC WEB

Software-Agenten holen automatisch: Behandlungsdaten · Versicherungsstatus · Therapeutenbewertungen · freie Termine · Kalender.

Das braucht semantisch strukturierte Daten — Maschinen müssen **Bedeutung** verstehen, nicht nur Strings vergleichen.

DAS KAPITEL IN EINEM BILD

🕒 1 Minute · Durchdenken

**Das Web verlinkt Dokumente**

Webseiten verweisen zwar aufeinander, doch ihre eigentliche **Bedeutung** (was der Inhalt aussagt) bleibt für Maschinen unsichtbar.

**Relationale DBs = Silos**

Feste Tabellen-Schemata erzwingen eine starre Struktur, wodurch das Zusammenführen von Daten aus verschiedenen Systemen aufwändig wird.

**RDF: Wissen als Graph**

RDF speichert Wissen als einzelne Fakten (Tripel), die sich frei miteinander verknüpfen lassen, auch über die Grenzen einzelner Datenquellen hinweg.

**Semantic Web & KI**

Weil die Bedeutung explizit modelliert ist, können Maschinen Inhalte tatsächlich interpretieren, statt nur Zeichenketten zu vergleichen (Grundlage für Semantic Web und KI).



Das **Warum** steht (klassische DBs stoßen an Grenzen). Als Nächstes im **Conceptual Model**: was genau *ist* ein Triple?

01

02

03

04

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 ARCHITECTURE

6 COMPARISON

WORUM ES IN DIESEM KAPITEL GEHT



Das Triple – Subjekt · Prädikat · Objekt als kleinste Wissenseinheit



URI · Literal · Blank Node – die Bausteine eines Graphen

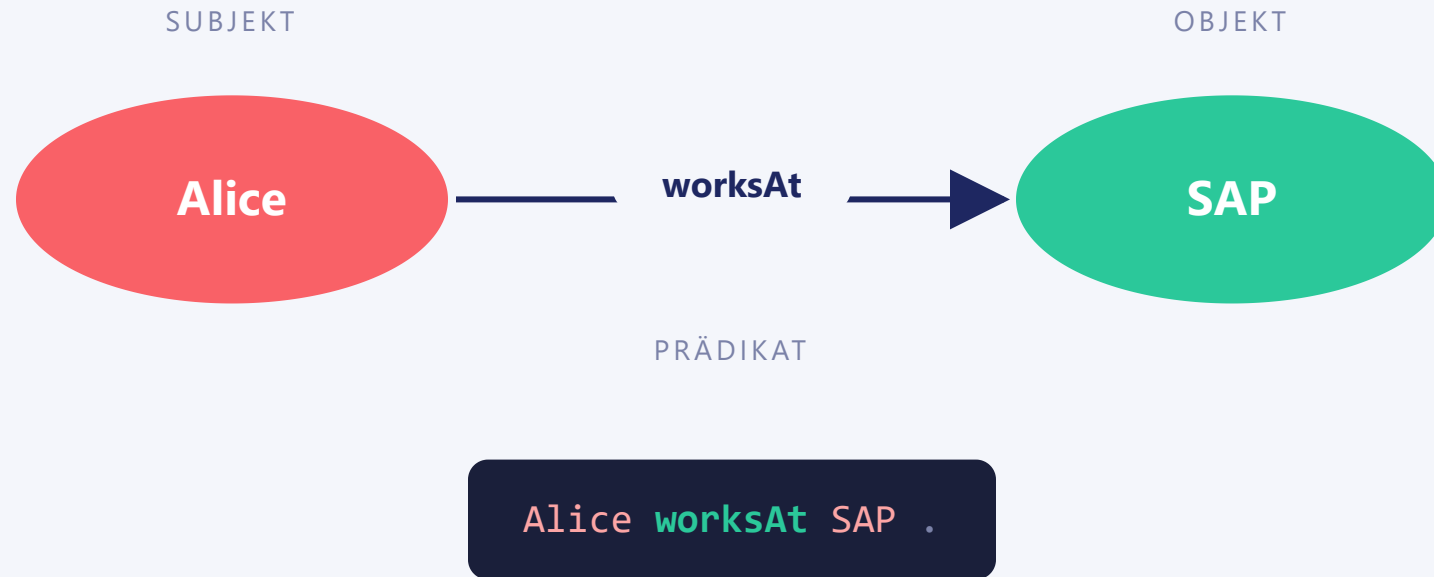


RDFS & OWL – Ontologien als Bauplan, *Inferenz* leitet neues Wissen ab



Named Graphs – mehrere Quellen vereinen & Kontext geben

DIE KLEINSTE AUSSAGE: SUBJEKT – PRÄDIKAT – OBJEKT



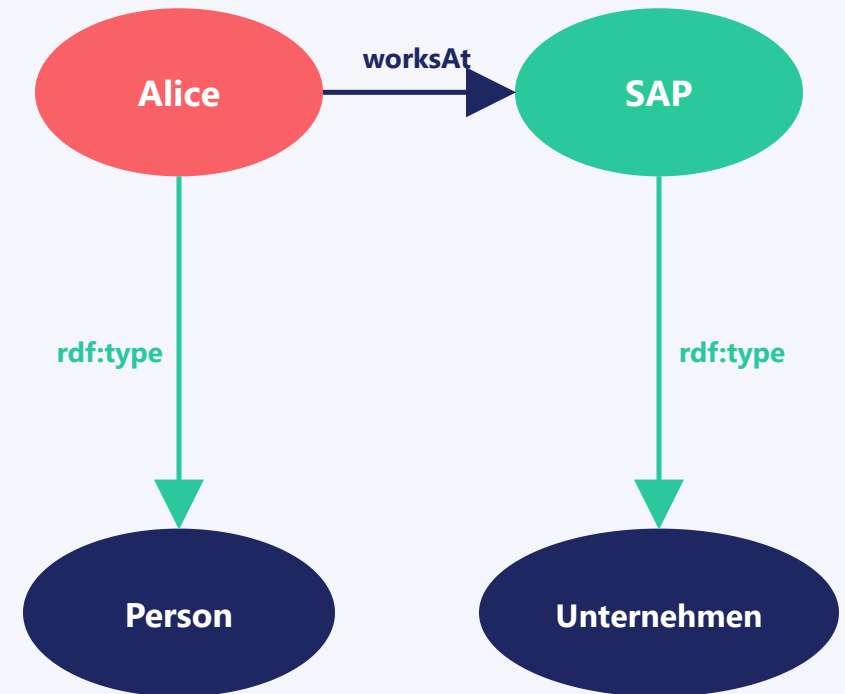
Ein Fakt = ein gerichteter Pfeil im Graph.

1 TRIPLE = 1 FAKT



Mehrere Triples über gemeinsame Knoten verschmelzen automatisch zu einem Graph.

MEHRERE FAKTEN = WISSENSGRAPH



PROBLEM: DASSELBE WORT, ZWEI DINGE

**„Apple“**

die Frucht

**„Apple“**

die Marke

? Welche „Apple“ ist gemeint — für Mensch *und* Maschine?

LÖSUNG: URI / IRI MACHT EINDEUTIG

`<dbpedia.org/resource/Apple>`

→ die Frucht

`<dbpedia.org/resource/Apple_Inc.>`

→ das Unternehmen

- ✓ Weltweit eindeutig & dereferenzierbar
- ✓ Kann Subjekt *und* Objekt sein → verknüpft Graphen

RESSOURCE (URI)



Eindeutige **Identität** eines Dings — global referenzierbar.

Aber: Nicht jeder Wert braucht eine eigene Identität — eine Jahreszahl, ein Name ...

LITERAL – KONKRETE DATENWERTE

```
:UniStuttgart gegrundet "1829"^^xsd:integer .  
:UniStuttgart rdfs:label "Universität Stuttgart"@de .
```

Typisiert: `xsd:integer` · `xsd:date` · `xsd:boolean`

🔗 Sprach-Tags: "Stuttgart"@de

→ Steht **immer nur als Objekt** — nie als Subjekt

ZWEI TYPISCHE ANWENDUNGSFÄLLE



ZUSAMMENGESETZTER WERT

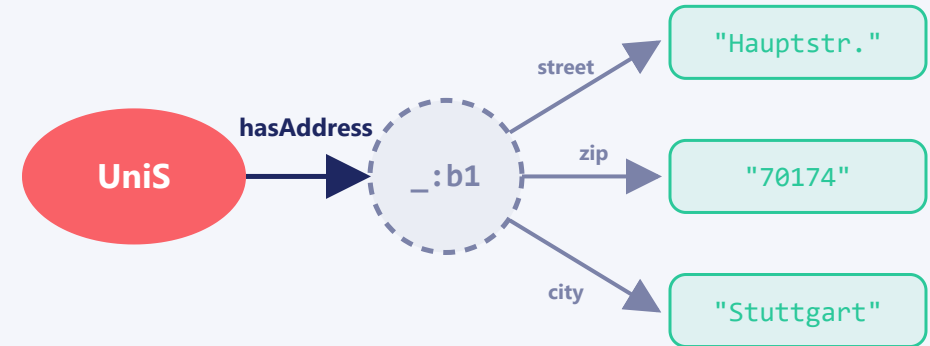
Eine Adresse (Straße + PLZ + Stadt) gehört zusammen, braucht aber keine eigene globale URI.



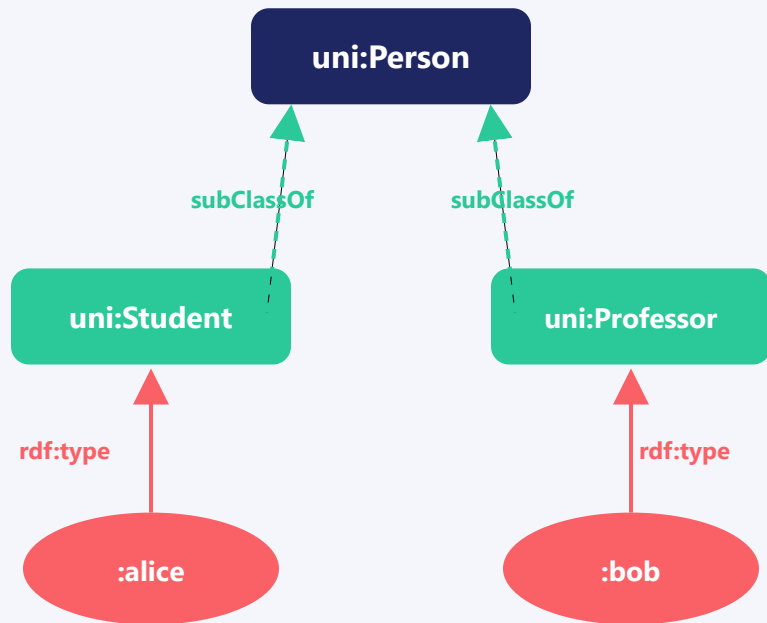
EXISTENZ OHNE IDENTITÄT

„Alice hat einen Betreuer“ — er *existiert*, ist aber (noch) nicht benannt.

LÖSUNG: BLANK NODE ALS ZWISCHENKNOTEN



- ⊘ Keine globale URI · nur lokal gültig (`_:b1`)
- ⊘ Von außen nicht referenzierbar



VOKABULAR: KLASSEN & EIGENSCHAFTEN DEFINIEREN

```
uni:Professor rdfs:subClassOf uni:Person .  
uni:worksAt rdfs:domain uni:Person .  
uni:worksAt rdfs:range uni:Company .
```

-  **subClassOf** baut Hierarchien — jeder Professor ist eine Person
-  **domain / range** legen erlaubte Typen von Subjekt & Objekt fest

Wozu? Kontrolliertes Vokabular → Basis für automatische Schlüsse.

OWL — ERWEITERTE ONTOLOGIE-REGELN

**OWL:EQUIVALENTCLASS**

Zwei Klassen = dasselbe Konzept

z.B. *Dozent* = *Professor*

```
uni:Dozent owl:equivalentClass uni:Professor .
```

**OWL:INVERSEOF**

teacherOf $\rightleftharpoons$ studentOf automatisch

```
uni:lehrt owl:inverseOf uni:lerntBei .
```

**OWL:SAMEAS**

Lokale URI = globale URI (DBpedia)

z.B. *uni:Einstein* = *dbpedia:Einstein*

```
uni:Einstein owl:sameAs dbr:Albert_Einstein .
```

INFERENZ: WISSEN ABLEITEN

Gespeichert:

```
uni:Student rdfs:subClassOf uni:Person .  
:alice a uni:Student .
```

REASONER

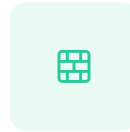
Automatisch abgeleitet:

```
:alice a uni:Person . ← NEU
```

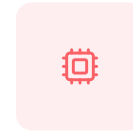

DREI EBENEN WIE DER BAU EINES HAUSES

**ONTOLOGIE**

Der **Bauplan** für das Haus —
was es gibt und wie es
zusammenhängt.

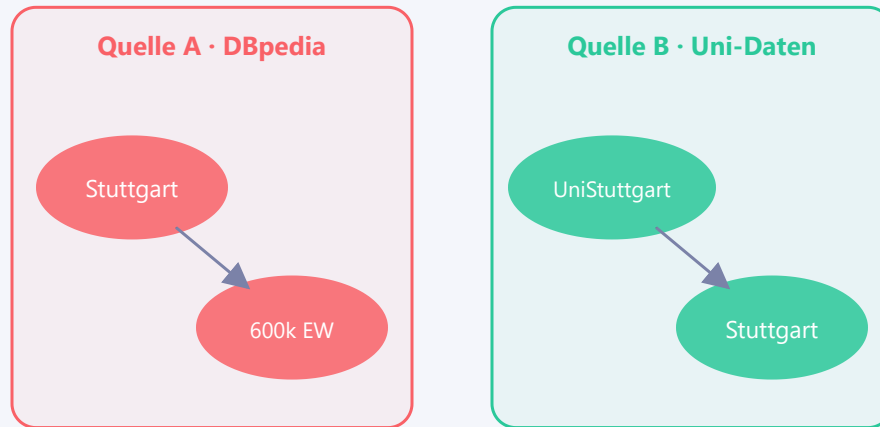
**RDFS**

Der einfache **Rohbau** — Wände
und Türen: Klassen &
Hierarchien.

**OWL**

Der detaillierte **Innenausbau** —
Elektronik & Logikschaltungen.

PROBLEM: ZWEI DATENQUELLEN ZUSAMMENFÜHREN



Im selben Store — aber **woher** stammt welches Tripel?

LÖSUNG: JEDER QUELLE IHREN EIGENEN GRAPH-NAMEN

```
GRAPH <graph:UniDaten> {  
  :UniStuttgart locatedIn :Stuttgart .  
}
```

→ 4. Element pro Tripel: der **Kontext** (welcher Graph).



Wofür kann man Named Graphs sonst noch nutzen?

EIN TRIPEL + KONTEXT

```
GRAPH <graph:UniDaten> {  
  :UniStuttgart locatedIn :Stuttgart .  
}
```

Der Graph-Name macht aus jedem Tripel ein **Quad** (S · P · O · Kontext).

EINSATZGEBIETE



PROVENIENZ

Woher stammt ein Tripel? Welche Quelle, welcher Zeitpunkt?



ZUGRIFFSRECHTE

Graphen einzeln absichern oder veröffentlichen



VERSIONIERUNG

Stände getrennt halten — alt vs. neu im selben Store

DAS KAPITEL IN EINEM BILD

🕒 1 Minute · Durchdenken

**Das Triple: S · P · O**

Jede Aussage besteht aus Subjekt, Prädikat und Objekt und bildet damit die kleinste, in sich geschlossene Wissenseinheit.

**Drei Bausteine**

Ressourcen werden über **URIs** eindeutig benannt, konkrete Werte stehen als **Literale** (Text, Zahl, Datum) und anonyme Hilfsknoten (**Blank Nodes**) bündeln Aussagen ohne eigene URI.

**Tripel bilden einen Graph**

Sobald mehrere Tripel dieselbe URI verwenden, verbinden sie sich automatisch zu einem durchgehenden Wissensnetz (ganz ohne Fremdschlüssel).

**RDFS & OWL**

Mit Schema-Sprachen lassen sich Klassen, Hierarchien und ganze Ontologien beschreiben, was die Grundlage für maschinelle **Inferenz** bildet.

**Named Graphs**

Tripel lassen sich einem benannten Graphen zuordnen, sodass sich Herkunft und Kontext mehrerer Quellen sauber auseinanderhalten lassen.



Das **Was** ist geklärt (Tripel, URIs, Ontologien). Als Nächstes die **Data Structure**: wie wird das alles gespeichert?

01

02

03

04

05

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 ARCHITECTURE

6 COMPARISON

WORUM ES IN DIESEM KAPITEL GEHT



Weg von der Tabelle – Speicherung als Triple statt Zeilen & Spalten



Dictionary-Encoding – URIs & Literale als kompakte IDs



Index-Permutationen – SPO · POS · OSP



Integration über URIs – globale Verknüpfung von Fakten

SQL — BREITE TABELLE MIT NULLS

NAME	UNI	REKTOR	GRÜNDUNG	WEBSITE
Stuttgart	UniS	Müller	NULL	uni-s.de
Karlsruhe	KIT	NULL	1825	NULL



Jede NULL-Zelle = verschwendeter Speicher. Schema muss vollständig sein.

RDF — NUR EXISTIERENDE FAKTEN

:UniStuttgart	uni:rektor	"Prof. Müller"
:UniStuttgart	uni:website	"uni-s.de"
:KIT	uni:gründung	"1825"



Kein Fakt vorhanden? Kein Triple — kein Platz verschwendet.

SQL — SCHEMA-ÄNDERUNG NÖTIG

```
-- AI-Agent findet neue Info:  
-- "Prof. Müller forscht an Quanten"
```

```
ALTER TABLE Professoren  
ADD COLUMN forschung VARCHAR(255);
```

```
-- → alle anderen Zeilen: NULL  
-- → bestehende Apps können brechen
```



Starr · hoher Verwaltungsaufwand · Risiko für laufende Systeme

RDF — EINFACH EIN TRIPLE ANHÄNGEN

```
-- AI-Agent findet neue Info:  
-- Einfach neues Triple einfügen:
```

```
:ProfMüller uni:forscht  
    "Quantencomputing" .
```

```
-- Das war's. Kein Schema-Eingriff.
```



Schema-light · organisch erweiterbar

VON URIS ZU INTEGER-IDS

Dictionary	Triple Table
101 → dbr:Stuttgart	S P O
201 → uni:UniStuttgart	201 301 101
301 → uni:locatedIn	201 401 501
401 → uni:hasName	statt:
501 → "Univ. Stuttgart"	uni:UniStuttgart
	uni:locatedIn
	dbr:Stuttgart

WARUM DICTIONARY-ENCODING?



SCHNELLE VERGLEICHE

Integer-Vergleich statt String-Vergleich — deutlich schneller bei Millionen Triples



KONPAKTER SPEICHER

Lange URIs nur einmal im Dictionary — in der Triple Table nur IDs



TRADE-OFF

Zusätzliche Dictionary-Verwaltung — bei Lookups ist immer ein Decode-Schritt nötig

6 INDEX-PERMUTATIONEN

SPO

SOP

PSO

POS

OSP

OPS

DIESELBEN 4 TRIPEL — ALS INTEGER-IDS (NACH DICTIONARY-ENCODING)

(101, 502, 201) UniS · loc · Stuttgart

(305, 502, 202) UniFR · loc · Freiburg

(101, 503, 701) UniS · name · „Uni Stuttgart“

(305, 503, 702) UniFR · name · „Uni Freiburg“

SPO-INDEX — SORTIERT NACH $S \rightarrow P \rightarrow O$ $S \geq 305 ?$

(101, 502, 201)
(101, 503, 701)

(305, 502, 202)
(305, 503, 702)

POS-INDEX — SORTIERT NACH $P \rightarrow O \rightarrow S$ $P \geq 503 ?$

(502, 201, 101)
(502, 202, 305)

(503, 701, 101)
(503, 702, 305)

6 INDEX-PERMUTATIONEN

SPO

SOP

PSO

POS

OSP

OPS

SPARQL-PATTERN

BEISPIEL

NUTZT INDEX

dbr:UniS ?p ?o

Alle Fakten über UniStuttgart

?s uni:loc dbr:Stuttgart

Was liegt in Stuttgart?

?s ?p dbr:Stuttgart

Wer referenziert Stuttgart?

6 INDEX-PERMUTATIONEN

SPO

SOP

PSO

POS

OSP

OPS

TRADE-OFF



6× Speicherverbrauch für stabile Query-Performance



Jede SPARQL-Anfrage findet passenden Index — kein Full Scan

SPARQL-PATTERN

BEISPIEL

NUTZT INDEX

dbr:UniS ?p ?o

Alle Fakten über UniStuttgart

SPO

?s uni:loc dbr:Stuttgart

Was liegt in Stuttgart?

POS

?s ?p dbr:Stuttgart

Wer referenziert Stuttgart?

OSP

MEHRDEUTIGKEIT OHNE URI

"Stuttgart" — Stadt?
Nachname? String?

URI ALS KNOTENPUNKT

:UniS → locatedIn → dbr:Stuttgart (Objekt)
dbr:Stuttgart → locatedIn → dbr:BW (Subjekt)
→ Kette: UniS → Stuttgart → BW

EINDEUTIGKEIT MIT URI

<http://dbpedia.org/resource/Stuttgart>

INTERNE DATEN

```
:MeineUni uni:loc  
    dbr:Stuttgart .  
:MeineUni uni:name  
    "Meine Hochschule" .
```

DBPEDIA-DATEN

```
dbr:Stuttgart  
    dbo:population  
        "634,000" .  
dbr:Stuttgart  
    dbo:country  
        dbr:Germany .
```

←
dbr:Stuttgart
→
gemeinsame URI
= automatische
Verknüpfung

DAS KAPITEL IN EINEM BILD

🕒 1 Minute · Durchdenken

**Alles ist ein Tripel**

Statt Tabellen mit festen Spalten speichert ein Triple Store jede Information als eigene Zeile aus Subjekt, Prädikat und Objekt.

**Schema-Flexibilität**

Es gibt kein starres Schema im Voraus, sodass jederzeit neue Eigenschaften und Beziehungen dazukommen können (das Modell wächst organisch mit).

**Dictionary-Encoding**

Wiederkehrende URIs und Literale landen einmalig in einem Wörterbuch und werden intern nur noch als kurze Zahl-IDs gespeichert (spart Platz und beschleunigt Vergleiche).

**Index-Permutationen**

Dieselben Tripel werden in mehreren Sortierungen (SPO, POS, OSP und weitere) abgelegt, damit jedes Anfrage-Muster effizient bedient wird.

**Integration über URIs**

Weil URIs global eindeutig sind, lassen sich Fakten aus völlig verschiedenen Quellen ohne zusätzliches Mapping miteinander verknüpfen.



Die Speicherung ist geklärt (Tripel, Indizes, URIs). Als Nächstes das **Query Model**: wie fragen wir das alles ab?

01

02

03

04

05

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 ARCHITECTURE

6 COMPARISON

WURUM ES IN DIESEM KAPITEL GEHT



SPARQL – Pattern Matching statt Tabellen-Joins



Query-Typen – SELECT · ASK · CONSTRUCT · DESCRIBE



FILTER · OPTIONAL · Aggregation · Property Paths

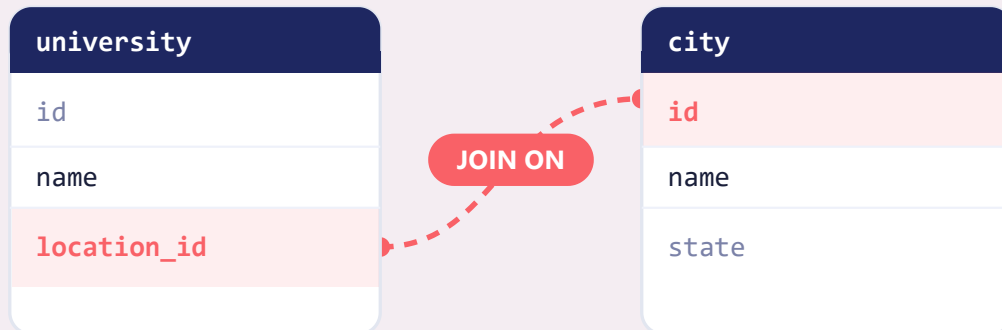


Inferenz – implizites Wissen abfragen



SQL — TABELLEN & EXPLIZITE JOINS

```
SELECT u.name, c.name
FROM university u
JOIN city c ON u.location_id = c.id
WHERE c.state = 'Baden-Württemberg'
```



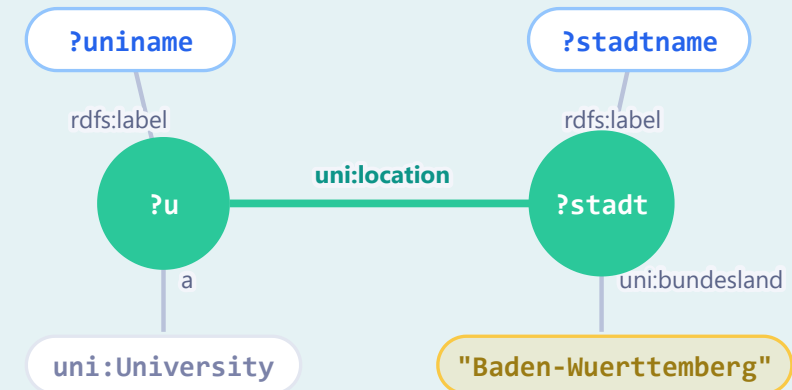
Tabellenschema nötig

Explizite JOIN-Keys



SPARQL — GRAPH PATTERN MATCHING

```
SELECT ?uniname ?stadtname
WHERE {
  ?u a uni:University .
  ?u rdfs:label ?uniname .
  ?u uni:location ?stadt .
  ?stadt uni:bundesland "Baden-Wuerttemberg" .
  ?stadt rdfs:label ?stadtname .
}
```



Kein JOIN-Keyword

Geteilte Variablen

W3C-Standard

ANATOMIE EINER SPARQL-QUERY

```
PREFIX uni: <http://example.org/uni/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?u ?name ← Ausgabe-Variablen
WHERE {
  ?u a uni:University . ← Typ-Pattern
  ?u rdfs:label ?name . ← Attribut-Pattern
  ?u uni:location ?stadt .
  ?stadt uni:bundesland "Baden-Wuerttemberg" .
}
ORDER BY ?name
```

?u ist dieselbe Variable in beiden Patterns — das verbindet sie wie ein JOIN, ohne JOIN-Keyword.

SCHLÜSSELKONZEPTE

 VARIABLEN MIT ?

Platzhalter für beliebige Werte — Typ entsteht durch Patterns, nicht durch den Namen

 PRÄFIXE

Kurzschreibweise für Namespace-URLs

 IMPLIZITES AND

Selektivstes Pattern zuerst (Optimizer)



SELECT — TABELLE

```
SELECT ?u ?name WHERE { ... }
```

Eine Spalte pro Variable, eine Zeile pro Match — Standardfall für alle Abfragen.



ASK — BOOLEAN

```
ASK { uni:AliceSchmidt a uni:Student . }
```

Nur **true** oder **false** — kein Datensatz. Validierung, Existenz-Checks.



CONSTRUCT — NEUER GRAPH

```
CONSTRUCT {  
  ?p foaf:name ?name ;  
    foaf:memberOf uni:Stuttgart .  
}  
WHERE {  
  ?p a uni:Student ;  
    uni:hatName ?name ;  
    uni:studiertAn uni:Stuttgart .  
}
```

Übersetzt uni:-Vokabular in foaf: — Quellen vereinheitlichen, RAG-Kontext aufbereiten.



DESCRIBE — RESSOURCE

```
DESCRIBE uni:Stuttgart
```

Alles über eine URI — store-definiert, oft äquivalent zu `SELECT ?p ?o WHERE { uni:Stuttgart ?p ?o }`. Schnelle Exploration.

PERSONEN PRO UNIVERSITÄT ZÄHLEN

```
SELECT ?uniname
  (COUNT(?person) AS ?anzahl)
WHERE {
  ?person uni:studiesAt ?u .
  ?u rdfs:label ?uniname .
}
GROUP BY ?uniname
ORDER BY DESC(?anzahl)
```

WAS STECKT DRIN?



AGGREGATION

COUNT(?person) zählt Personen pro Gruppe ·
GROUP BY ?uniname gruppiert nach Universität
— Syntax wie SQL GROUP BY

FILTER

≈ SQL WHERE

```
SELECT ?name ?sem
WHERE {
  ?s a uni:Student;
     rdfs:label ?name;
     uni:semester ?sem.
  FILTER(?sem >= 5)
}
```



Bedingung auf Ergebnisse

Schränkt **bereits gebundene** Variablen ein

OPTIONAL

≈ LEFT JOIN

```
SELECT ?name ?fach
WHERE {
  ?p a uni:Professor;
     rdfs:label ?name.
  OPTIONAL {
    ?p uni:fach ?fach.
  }
}
```



Optional statt Pflicht

Fehlendes Triple = „**unbekannt**“ (Open World). Ohne OPTIONAL fällt die Zeile raus — **0 Treffer, kein Fehler**.

PROPERTY PATH

Graph-Traversal

```
-- Alternative Prädikate:
?person (uni:studiesAt
        | uni:worksAt) ?u .

-- Transitiv (0+ Schritte):
?typ rdfs:subClassOf* uni:Person .
```



Traversal in einer Zeile

`subClassOf*` = beliebig viele Hops die Hierarchie hoch — die Engine läuft sie ab, **keine Schleife im Code**.

| oder · * 0+ · + ≥1 · / A→B

**OHNE INFERENZ**

```
SELECT ?p WHERE { ?p a uni:Person . }
```

0**ERGEBNISSE**

Niemand ist explizit als uni:Person gespeichert

MIT RDFS:SUBCLASSOF*

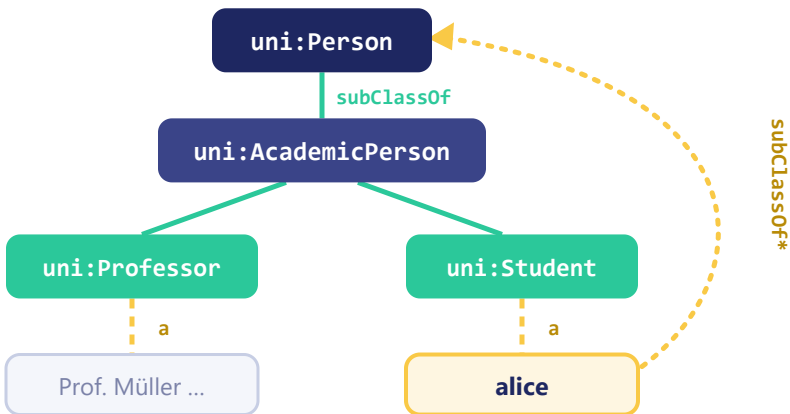
```
SELECT ?p WHERE {  
  ?p a ?typ .  
  ?typ rdfs:subClassOf* uni:Person .  
}
```

**REASONER
AKTIVIEREN****7****ERGEBNISSE**

Studenten + Professoren + Staff werden erkannt



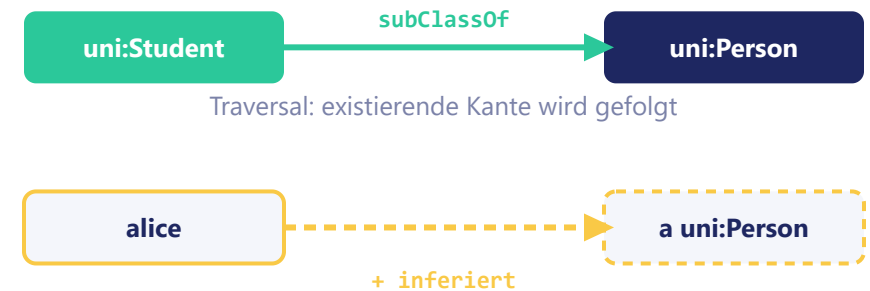
KLASSENHIERARCHIE IM DATENSATZ



Gespeichert ist nur `alice a uni:Student`. Über die Kette **Student** → **AcademicPerson** → **Person** macht `subClassOf*` sie transitiv zur **uni:Person**



TRAVERSAL VS. INFERENZ



Hier: **Traversal** entlang gespeicherter `subClassOf`-Kanten. Bei **Inferenz/Entailment** (Architecture) leitet der Store das neue Triple ab und speichert es zusätzlich.

SERVICE — LIVE-ANFRAGE GEGEN WIKIDATA

```

SELECT ?uniname ?description
WHERE {
  -- lokale Daten:
  ?u a uni:University .
  ?u rdfs:label ?uniname .
  ?u uni:location ?wdCity .

  -- externe Quelle (live):
  SERVICE <https://query.wikidata.org/sparql> {
    VALUES ?wdCity { wd:Q1726 wd:Q64 }
    ?wdCity schema:description ?description .
    FILTER (lang(?description) = "de")
  }
}

```

Standort zeigt direkt auf eine globale **Wikidata**-URI — kein Datenkopie nötig

WARUM GEHT DAS NUR MIT W3C-SPARQL?



 Oxigraph, DBpedia, Wikidata, GraphDB, Neptune — **alle sprechen denselben Standard**

 Neo4j Cypher, MongoDB: **keine** gemeinsame Abfragesprache mit externen Endpoints

 **Ein** Query-Mechanismus, **viele** Datenquellen weltweit

AUSGABEFORMATE

SELECT JSON · CSV · TSV APIs, RAG, Tabellen

CONSTRUCT Turtle · RDF/XML · JSON-LD Graph-Export

DAS KAPITEL IN EINEM BILD

🕒 1 Minute · Durchdenken

**SPARQL = Pattern Matching**

Man beschreibt ein gesuchtes **Graph-Muster**, woraufhin die Engine alle dazu passenden Belegungen findet (ganz ohne ausformulierte Joins).

**Geteilte Variablen verbinden**

Taucht dieselbe Variable (etwa ?u) in mehreren Patterns auf, verknüpft sie diese automatisch und wirkt damit wie ein impliziter **Join**.

**Vier Ergebnisformen**

Je nach Ziel liefert SPARQL eine Tabelle (**SELECT**), ein Ja/Nein (**ASK**), einen neuen Graphen (**CONSTRUCT**) oder alle Fakten zu einer Ressource (**DESCRIBE**).

**Verfeinern & Traversieren**

FILTER schränkt Ergebnisse über Bedingungen ein, **OPTIONAL** lässt fehlendes Wissen zu (Open World) und **Property Paths** traversieren den Graphen, ohne dass man Schleifen programmiert.

**Inferenz (implizites Wissen)**

SPARQL liefert nicht nur wörtlich gespeicherte Fakten, sondern auch das, was im Graph logisch **gilt** (etwa Unterklassen, die über die Klassenhierarchie automatisch mitgefunden werden).



Das Abfragen sitzt (SPARQL, Inferenz, Federation). Als Nächstes die **Architecture**: wie läuft das produktiv und skalierbar?

01

02

03

04

05

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 **ARCHITECTURE**

6 COMPARISON

WORUM ES IN DIESEM KAPITEL GEHT



Vom Modell zur Anwendung – Storage-Engines & Tools
(Oxigraph ...)



Deployment – selbst betreiben vs. Cloud



Skalierung – Single Node vs. Distributed, Replication & Sharding



RDF-Skalierungsproblem – verteilte Graph-Joins & Partitionierung

TYPISCHE ANWENDUNGSFÄLLE

**SEMANTISCHE UNTERNEHMENSsuche**

„Wer arbeitet an SAP-Projekten im Bereich Logistik?“ — RDF verknüpft Mitarbeiter, Projekte, Fachbereiche

**RAG-SYSTEME MIT LLMS**

SPARQL liefert strukturierten Kontext — LLM beantwortet Fragen mit nachvollziehbaren Quellen

**KNOWLEDGE GRAPHS / SUCHMASCHINEN**

Verknüpfung von Personen, Orten, Ereignissen — Zusammenhänge statt isolierter Datensätze



SELBST BETREIBEN

VS.



CLOUD-DIENST

**Wann betreibt man einen Triple Store selbst,
wann nutzt man einen Cloud-Anbieter?**

Welche Vor- und Nachteile bringt jede Variante mit sich?

**SELBST
BETREIBEN**

- + Kontrolle, geringe Kosten
- eigener Betriebsaufwand

**CLOUD-
DIENST**

- + kein Aufwand, skaliert
- Kosten, Anbieterbindung

OPEN SOURCE**Oxigraph**

Leichtgewichtig · Rust

Einsatz: Demo, kleine Apps

Apache Jena Fuseki

Reasoning-Engine · einfacher Einstieg

Einsatz: Lehre, Forschung

CLOUD / ENTERPRISE**GraphDB**

Reasoning · Linked Data Management:

Einsatz: Unternehmensprojekte

Amazon Neptune

Auto-Skalierung · Multi-Model

Einsatz: große Produktivsysteme

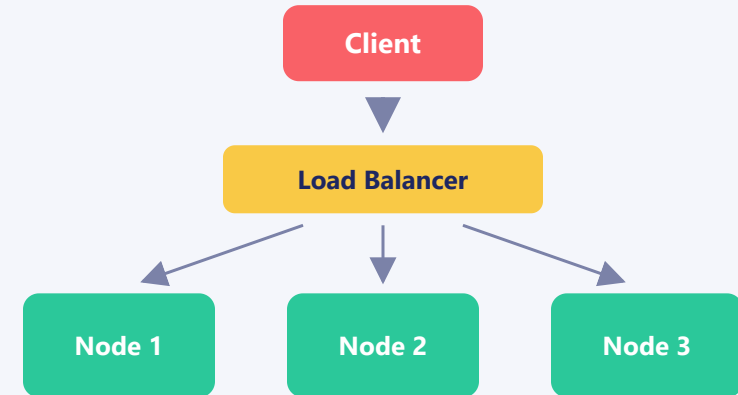
SINGLE NODE



- ✓ Einfache Einrichtung & Betrieb
- ✓ Keine Konsistenzprobleme
- ≈ Nur vertikale Skalierung (CPU/RAM/SSD)
- ✗ Skalierungsgrenze auf einer Maschine

Jena TDB2 · GraphDB Free · Oxigraph

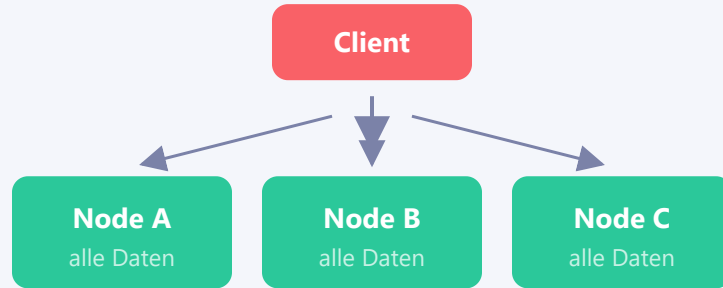
DISTRIBUTED



- ✓ Horizontale Skalierung & hohe Verfügbarkeit
- ✗ Höhere Komplexität
- ✗ Verteilte SPARQL-Joins über Node-Grenzen

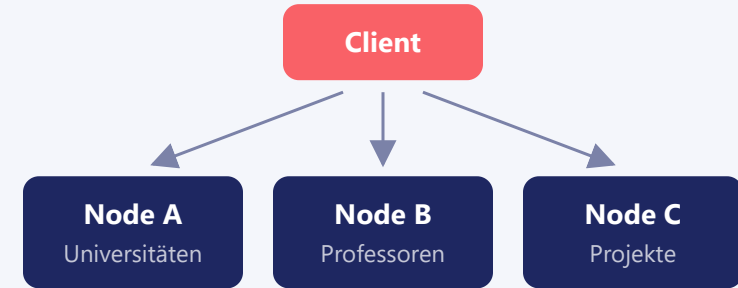
GraphDB Cluster · Amazon Neptune

REPLICATION — GLEICHE DATEN MEHRFACH



- ✓ Ausfallsicherheit
- ✓ Lastverteilung (Lesezugriffe)

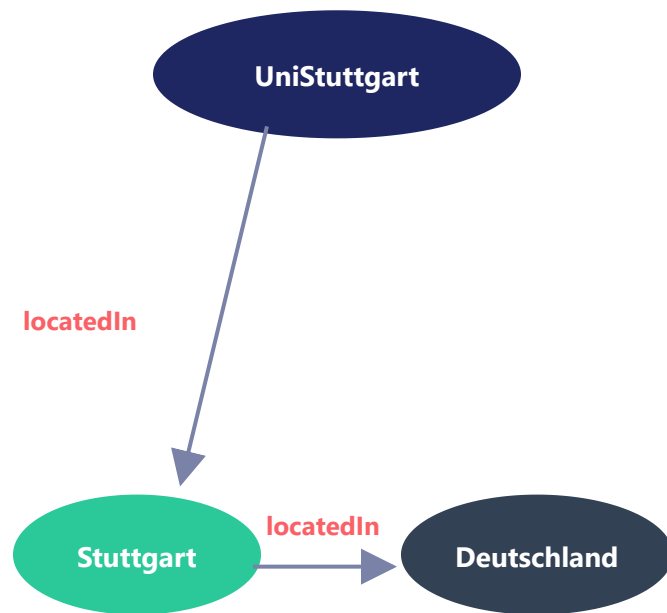
SHARDING — DATEN AUFTEILEN



- ✓ Mehr Gesamtspeicher
- ✓ Parallele Verarbeitung

CAP-Theorem: System: Zwei der drei Eigenschaften gleichzeitig garantieren: **Consistency, Availability, Partition Tolerance**

ZUSAMMENHÄNGENDER RDF-GRAPH



DATEN AUF VERSCHIEDENEN NODES

NODE A

UniStuttgart locatedIn Stuttgart

NODE B

Stuttgart locatedIn Deutschland

→
PARTITIO-
NIERUNG

PROBLEM BEI SPARQL-JOIN

Kette UniS → Stuttgart → Deutschland braucht einen Netzwerk-Hop zwischen Node A und B.

**SUBJECT-BASIERT**

Alle Tripel eines Subjekts auf denselben Node

UniStuttgart

hasName
locatedIn

UniKarlsruhe

hasName
founded

- ✓ Einfach umzusetzen
- ✗ Cross-Subjekt-Joins bleiben



Einsatz: Entitäts-zentrierte
Abfragen

**PREDICATE-BASIERT**

Aufteilung nach Prädikat — alle gleichen Kanten auf einem Node

Node A

alle locatedIn

Node B

alle worksAt

- ✓ Gut für Analyse-Workloads
- ✗ Multi-Prädikat-Queries teuer



Einsatz: analytische Statistik-
Abfragen

**GRAPH PARTITIONING**

Zusammenhängende Teilgraphen gemeinsam auf einem Node

Node A

Universitäten
+ Professoren

Node B

Unternehmen
+ Mitarbeiter

- ✓ Weniger Cross-Node-Joins
- ✗ Komplexe Vorberechnung nötig



Einsatz: stark vernetzte Domänen

DAS KAPITEL IN EINEM BILD

🕒 1 Minute · Durchdenken

**Storage-Engines**

Spezialisierte Engines (zum Beispiel **Oxigraph**) übernehmen das Speichern und Indexieren der Tripel.

01

**Deployment**

Ein Triple Store lässt sich entweder selbst betreiben oder als fertiger **Managed-Cloud-Dienst** beziehen.

02

**Skalierung**

Replication erhöht die Verfügbarkeit (mehrere Kopien), während **Sharding** die Daten auf mehrere Knoten verteilt.

03

**Partitionierung**

Die Verteilung kann nach Subjekt, Prädikat oder zusammenhängenden Teilgraphen erfolgen, je nach erwartetem Workload.

04

**Verteilte Graph-Joins**

Die zentrale Herausforderung beim Sharding: Tripel sind über den ganzen Graph verflochten, sodass Joins über Knotengrenzen hinweg teuer werden.

05



Der Betrieb ist klar (Engines, Skalierung, Sharding). Als Nächstes der **Comparison**: Triple Store gegen SQL und Property Graph.

1 MOTIVATION

2 CONCEPTUAL MODEL

3 DATA STRUCTURE

4 QUERY MODEL

5 ARCHITECTURE

6 COMPARISON

WORUM ES IN DIESEM KAPITEL GEHT



vs. SQL – Schema-Flexibilität & Graph-Joins



vs. Property Graph – Standards, Inferenz, globale URIs



Stärken & Schwächen im direkten Überblick



Use-Case-Matrix – wann nimmt man was?

Zeile in Tabelle ✓	 DATENEINHEIT	✓ Subjekt – Prädikat – Objekt
SQL ✓	 ABFRAGESPRACHE	✓ SPARQL (Pattern Matching)
garantiert ✓	 KONSISTENZ	≈ begrenzt
starr, vordefiniert ≈	 SCHEMA	✓ flexibel, schema-light
Joins (komplex) ✗	 BEZIEHUNGEN	✓ Geteilte Variablen
keine ✗	 SEMANTIK	✓ RDFS / OWL, inferenzfähig
schwierig (ETL) ✗	 DATENINTEGRATION	✓ URIs → direkt verlinkbar

vendor-spezifisch (Cypher)




STANDARDISIERUNG


W3C-Standard · SPARQL, OWL

manuell implementieren




INFERENZ


RDFS / OWL built-in



KANTEN-PROPERTIES

pointer-nativ, sehr schnell




**TRAVERSAL-
PERFORMANCE**


Index-basiert

Insellösung




DATENINTEGRATION


URIs · Linked Data · Federation

vendor-spezifisch (Cypher)


STANDARDISIERUNG

W3C-Standard · SPARQL, OWL

manuell implementieren


INFERENZ

RDFS / OWL built-in

nativ an der Kante


KANTEN-PROPERTIES

nicht nativ möglich

pointer-nativ, sehr schnell


TRAVERSAL-
PERFORMANCE

Index-basiert

Insellösung


DATENINTEGRATION

URIs · Linked Data · Federation

**STABILE STRUKTUR****TIEFE GRAPHNAVIGATION****SEMANTIK & INFERENZ****OFFENE DATENINTEGRATION****EINSTIEG & TOOLING****KNOWLEDGE GRAPHS**

**STABILE STRUKTUR****Relational****TIEFE GRAPHNAVIGATION****SEMANTIK & INFERENZ****OFFENE DATENINTEGRATION****EINSTIEG & TOOLING****KNOWLEDGE GRAPHS**



STABILE STRUKTUR



Relational



TIEFE GRAPHNAVIGATION



Property Graph



SEMANTIK & INFERENZ



OFFENE DATENINTEGRATION



EINSTIEG & TOOLING



KNOWLEDGE GRAPHS





STABILE STRUKTUR



Relational



TIEFE GRAPHNAVIGATION



Property Graph



SEMANTIK & INFERENZ



Triple Store



OFFENE DATENINTEGRATION



EINSTIEG & TOOLING



KNOWLEDGE GRAPHS





STABILE STRUKTUR



Relational



TIEFE GRAPHNAVIGATION



Property Graph



SEMANTIK & INFERENZ



Triple Store



OFFENE DATENINTEGRATION



Triple Store



EINSTIEG & TOOLING



KNOWLEDGE GRAPHS





STABILE STRUKTUR



Relational



TIEFE GRAPHNAVIGATION



Property Graph



SEMANTIK & INFERENZ



Triple Store



OFFENE DATENINTEGRATION



Triple Store



EINSTIEG & TOOLING



Relational

Property Graph



KNOWLEDGE GRAPHS





STABILE STRUKTUR



Relational



TIEFE GRAPHNAVIGATION



Property Graph



SEMANTIK & INFERENZ



Triple Store



OFFENE DATENINTEGRATION



Triple Store



EINSTIEG & TOOLING



Relational

Property Graph



KNOWLEDGE GRAPHS



Triple Store

DAS GROSSE BILD

⚖️ Jedes Modell hat seinen Zweck

**Relational**

SQL · Tabellen

Stabile, bekannte Strukturen mit festem Schema und starken Garantien.

📺 z. B. Banking, ERP, Buchhaltung

Transaktionen

Reifes Tooling

**Property Graph**

Knoten · Kanten

Beziehungen als Kern — tiefe Pfad-Navigation und schnelle Traversierung.

👤 z. B. Social Networks, Routing, Fraud

Pfad-Queries

Netzwerke

**Triple Store**

RDF · SPARQL

UNSER FOKUS

Glänzt bei **Semantik & Inferenz** und offener Datenintegration über W3C-Standards.

🌐 z. B. Linked Data, Open Data

Inferenz

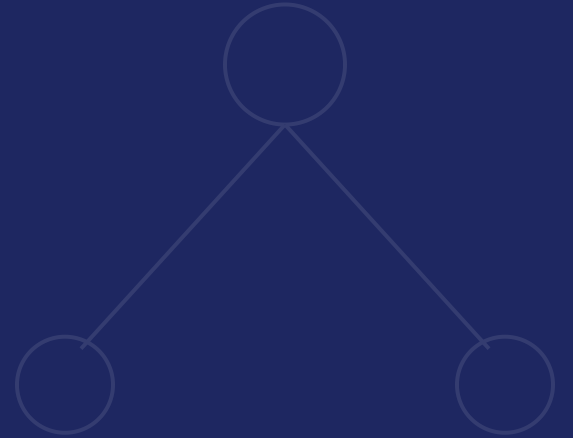
Standards

Knowledge Graphs

>_ Soweit die Theorie — jetzt sehen wir den Triple Store **live in der Demo**.

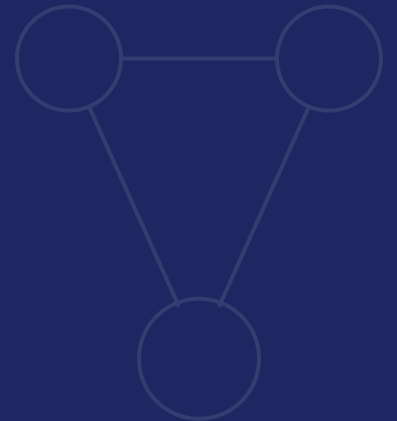


Demo





Interaktion





Quiz





Wer erfand 1989 am CERN das World Wide Web?



A

Vint Cerf

B

C

D



Wer erfand 1989 am CERN das World Wide Web?



A

Vint Cerf

B

Tim Berners-Lee

C

D



Wer erfand 1989 am CERN das World Wide Web?



A

Vint Cerf

B

Tim Berners-Lee

C

Robert Kahn

D



Wer erfand 1989 am CERN das World Wide Web?



A

Vint Cerf

B

Tim Berners-Lee

C

Robert Kahn

D

Marc Andreessen



Wer erfand 1989 am CERN das World Wide Web?



A

Vint Cerf

B

Tim Berners-Lee

RICHTIG

C

Robert Kahn

D

Marc Andreessen

Warum? Tim Berners-Lee schrieb 1989 am CERN den Vorschlag „Information Management“ — die Geburtsstunde des WWW.



Wonach suchen Pete & Lucy im Semantic-Web-Szenario (2001)?



A

Pizzeria

B

C

D



Wonach suchen Pete & Lucy im Semantic-Web-Szenario (2001)?



A

Pizzeria

B

Hotel

C

D



Wonach suchen Pete & Lucy im Semantic-Web-Szenario (2001)?



A

Pizzeria

B

Hotel

C

Physiotherapeut

D



Wonach suchen Pete & Lucy im Semantic-Web-Szenario (2001)?



A

Pizzeria

B

Hotel

C

Physiotherapeut

D

Gebrauchtwagen



Wonach suchen Pete & Lucy im Semantic-Web-Szenario (2001)?



A

Pizzeria

B

Hotel

C

Physiotherapeut

RICHTIG

D

Gebrauchtwagen

Warum? Pete & Lucy suchen einen Physiotherapeuten für ihre Mutter — das Leitbeispiel aus „The Semantic Web“ (2001).



Was darf in RDF NIE als Subjekt stehen?



A

URI

B

C

D



Was darf in RDF NIE als Subjekt stehen?



A

URI

B

Ressource

C

D



Was darf in RDF NIE als Subjekt stehen?



A

URI

B

Ressource

C

Literal

D



Was darf in RDF NIE als Subjekt stehen?



A

URI

B

Ressource

C

Literal

D

Blank Node



Was darf in RDF NIE als Subjekt stehen?



A URI

B Ressource

C Literal

RICHTIG

D Blank Node

Warum? Literale (konkrete Werte wie Zahlen oder Strings) stehen immer nur als Objekt — nie als Subjekt.



Welches OWL-Konstrukt macht aus teacherOf automatisch studentOf?



A

owl:sameAs

B

C

D



Welches OWL-Konstrukt macht aus `teacherOf` automatisch `studentOf`?

**A**`owl:sameAs`**B**`owl:equivalentClass`**C****D**



Welches OWL-Konstrukt macht aus `teacherOf` automatisch `studentOf`?

**A**`owl:sameAs`**B**`owl:equivalentClass`**C**`rdfs:subClassOf`**D**



Welches OWL-Konstrukt macht aus `teacherOf` automatisch `studentOf`?

**A**`owl:sameAs`**B**`owl:equivalentClass`**C**`rdfs:subClassOf`**D**`owl:inverseOf`



Welches OWL-Konstrukt macht aus `teacherOf` automatisch `studentOf`?



A

`owl:sameAs`

B

`owl:equivalentClass`

C

`rdfs:subClassOf`

D

`owl:inverseOf`

RICHTIG

Warum? `owl:inverseOf` verknüpft zwei Properties als Umkehrung — die Beziehung gilt automatisch in beide Richtungen.



Ein Named Graph macht aus einem Triple ein...?



A

Quad

B

C

D



Ein Named Graph macht aus einem Triple ein...?



A

Quad

B

Tupel

C

D



Ein Named Graph macht aus einem Triple ein...?



A

Quad

B

Tupel

C

Schema

D



Ein Named Graph macht aus einem Triple ein...?



A

Quad

B

Tupel

C

Schema

D

Literal



Ein Named Graph macht aus einem Triple ein...?



A

Quad

RICHTIG



B

Tupel

C

Schema

D

Literal

Warum? Subjekt – Prädikat – Objekt + Graph-Name (Kontext) = vier Elemente = Quad.



Pattern „?s uni:locatedIn dbr:Stuttgart” — welcher Index passt am besten?

**A**

SPO

B**C****D**



Pattern „?s uni:locatedIn dbr:Stuttgart“ — welcher Index passt am besten?

**A**

SPO

B

SOP

C**D**



Pattern „?s uni:locatedIn dbr:Stuttgart“ — welcher Index passt am besten?

**A**

SPO

B

SOP

C

POS

D



Pattern „?s uni:locatedIn dbr:Stuttgart" — welcher Index passt am besten?

**A**

SPO

B

SOP

C

POS

D

OSP



Pattern „?s uni:locatedIn dbr:Stuttgart” — welcher Index passt am besten?



A

SPO

B

SOP

C

POS

RICHTIG



D

OSP

Warum? Prädikat und Objekt sind bekannt, Subjekt ist gesucht → der POS-Index liefert direkten Zugriff ohne Full Scan.



Was speichert die Triple Table dank Dictionary-Encoding statt langer URIs?



A

Hashes

B

C

D



Was speichert die Triple Table dank Dictionary-Encoding statt langer URIs?

**A**

Hashes

B

Integer-IDs

C**D**



Was speichert die Triple Table dank Dictionary-Encoding statt langer URIs?

**A**

Hashes

B

Integer-IDs

C

Pointer

D



Was speichert die Triple Table dank Dictionary-Encoding statt langer URIs?



A

Hashes

B

Integer-IDs

C

Pointer

D

Strings



Was speichert die Triple Table dank Dictionary-Encoding statt langer URIs?



A Hashes

B Integer-IDs

RICHTIG

C Pointer

D Strings

Warum? URI steht einmal im Dictionary, die Triple Table enthält nur IDs → schnellere Vergleiche, kompakterer Speicher.



Was kann ein Property Graph nativ, ein RDF-Triple-Store nicht?



A

Inferenz

B

C

D



Was kann ein Property Graph nativ, ein RDF-Triple-Store nicht?

**A**

Inferenz

B

Standard-Abfrage

C**D**



Was kann ein Property Graph nativ, ein RDF-Triple-Store nicht?



A

Inferenz

B

Standard-Abfrage

C

URIs

D



Was kann ein Property Graph nativ, ein RDF-Triple-Store nicht?

**A**

Inferenz

B

Standard-Abfrage

C

URIs

D

Kanten-Attribute



Was kann ein Property Graph nativ, ein RDF-Triple-Store nicht?



A

Inferenz

B

Standard-Abfrage

C

URLs

D

Kanten-Attribute

RICHTIG

Warum? Property Graphs speichern Attribute direkt an Kanten — in RDF braucht das Reifikation bzw. RDF-star.



Was bedeutet das „*“ in `rdfs:subClassOf*`?



A

0 oder mehr Schritte

B

C

D



Was bedeutet das „*“ in `rdfs:subClassOf*`?



A

0 oder mehr Schritte

B

Genau 1 Schritt

C

D



Was bedeutet das „*“ in `rdfs:subClassOf*`?



A

0 oder mehr Schritte

B

Genau 1 Schritt

C

Multiplikation

D



Was bedeutet das „*“ in `rdfs:subClassOf*`?



A

0 oder mehr Schritte

B

Genau 1 Schritt

C

Multiplikation

D

Beliebiges Prädikat



Was bedeutet das „*“ in `rdfs:subClassOf*`?

**A****0 oder mehr Schritte****RICHTIG** **B****Genau 1 Schritt****C****Multiplikation****D****Beliebiges Prädikat**

Warum? In SPARQL Property Paths steht * für „0 oder mehr Schritte“ — die Engine läuft die ganze Hierarchie hoch.

W3C STANDARDS

- 1 **RDF 1.1 Concepts and Abstract Syntax** · W3C Recommendation, Feb. 2014
w3.org/TR/rdf11-concepts/
- 2 **RDF 1.1 Primer** · W3C Working Group Note, June 2014
w3.org/TR/rdf11-primer/
- 3 **SPARQL 1.1 Overview** · W3C Recommendation, March 2013
w3.org/TR/sparql11-overview/
- 4 **SPARQL 1.1 Query Language** · W3C Recommendation, March 2013
w3.org/TR/sparql11-query/
- 5 **SPARQL 1.1 Federated Query** · W3C Recommendation, March 2013
w3.org/TR/sparql11-federated-query/
- 6 **SPARQL 1.1 Entailment Regimes** · W3C Recommendation, March 2013
w3.org/TR/sparql11-entailment/
- 7 **Information Management: A Proposal** · Tim Berners-Lee, CERN, March 1989
w3.org/History/1989/proposal-msw.html

BILDER

- 8 **Tim Berners-Lee** · Encyclopædia Britannica
britannica.com/biography/Tim-Berners-Lee
- 9 **First Web Browser Screenshot** · CERN
info.cern.ch/images/screensnap2_24c.gif

LITERATUR & TOOLS

- 10 **Wie entstand das Internet?** · SWB Magazin
swb.de/ueber-swb/swb-magazin/swb-insider/wie-entstand-das-internet
- 11 **Meier, A.; Kaufmann, M.** SQL- & NoSQL-Datenbanken. Springer-Verlag, 8. Aufl. 2023.
[doi:10.1007/978-3-662-67092-7](https://doi.org/10.1007/978-3-662-67092-7)
- 12 **Gilbert, E.** Triplestores 101: Storing Data for Efficient Inferencing. Dataversity, 2016.
dataversity.net/articles/triplestores-101-storing-data-efficient-inferencing/

TOOL

- 13 **Oxigraph** – Rust-based SPARQL 1.1 triple store. Open source, kein JVM.
github.com/oxigraph/oxigraph
- 14 **Wikidata** – Freier Wissensgraph der Wikimedia Foundation. SPARQL-Endpoint für Federated Queries (Demo Query 7).
query.wikidata.org

VIDEOS

- 15 **An Introduction to Wikidata** · National Library of Wales
youtube.com/watch?v=m_9_23jXPoE
- 16 **Knowledge Graphs & Semantic Web**
youtube.com/watch?v=OGg8A2zfWKg